



Departamento de Lenguajes y
Sistemas Informáticos

Implementación de Servicios RESTful (I)

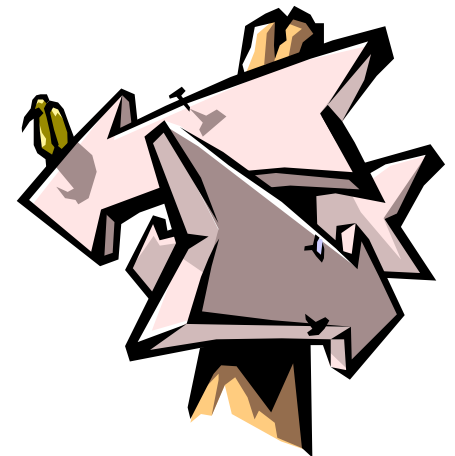
Integración



Arquitectura e Integración del Software

Contenido de la práctica

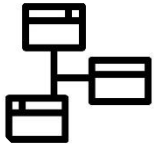
- **Vídeo 1:**
 - Estructura del proyecto de ejemplo
 - Creación del modelo
 - Operaciones de consulta:
- **Vídeo 2:**
 - Operaciones de creación
 - Operaciones de actualización
 - Operaciones de eliminación
- **Vídeo 3:**
 - Pruebas



Índice – Vídeo 1



Estructura del proyecto de ejemplo

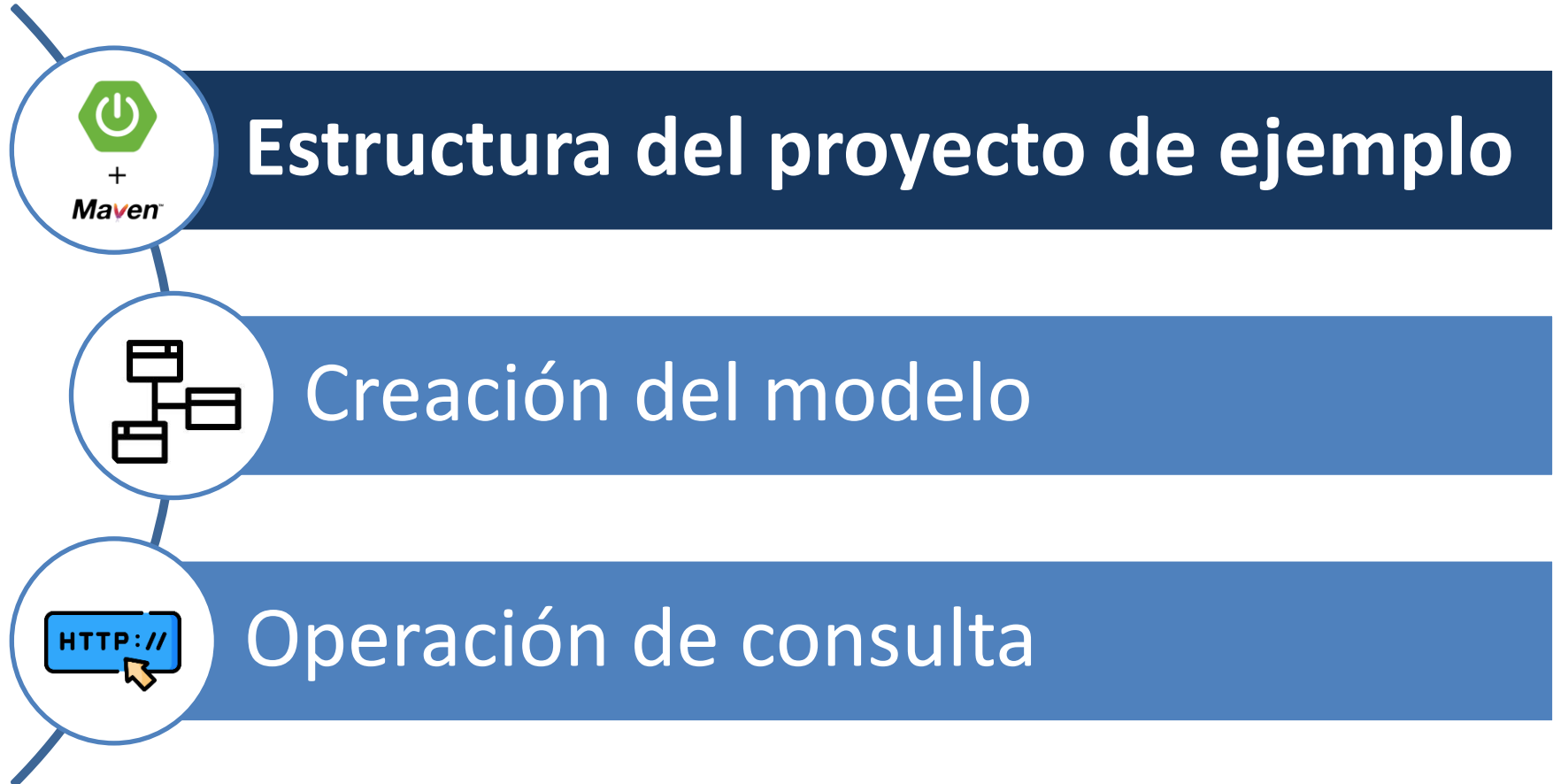


Creación del modelo



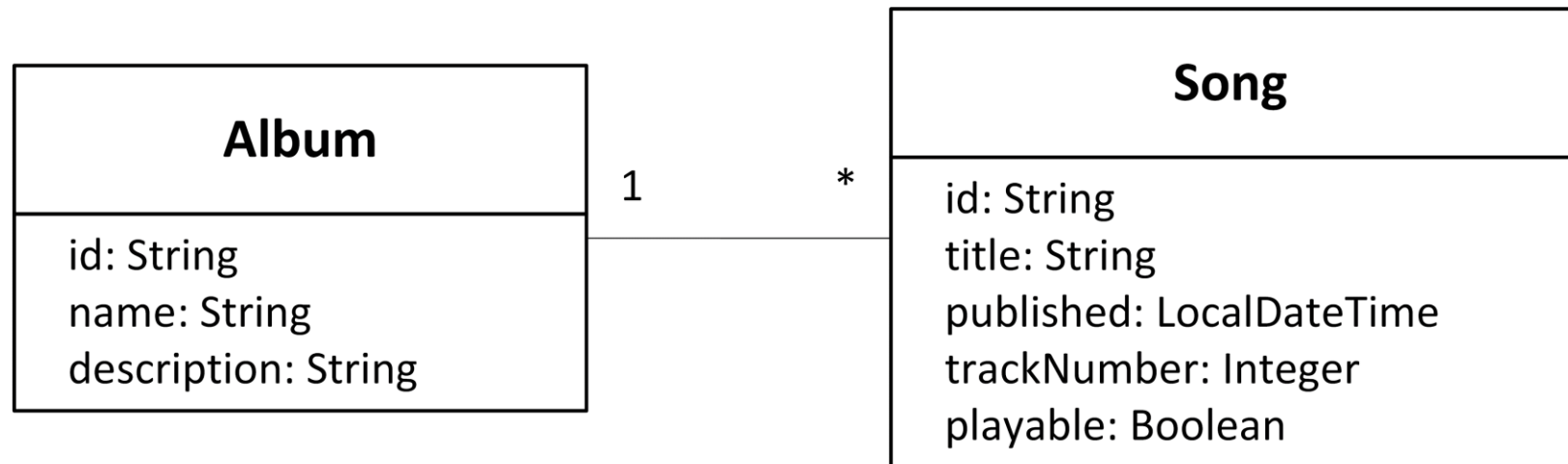
Operación de consulta

Índice – Vídeo 1



Estructura del proyecto de ejemplo

Modelo de datos



Estructura del proyecto de ejemplo

Operaciones sobre los recursos

Recurso Álbum

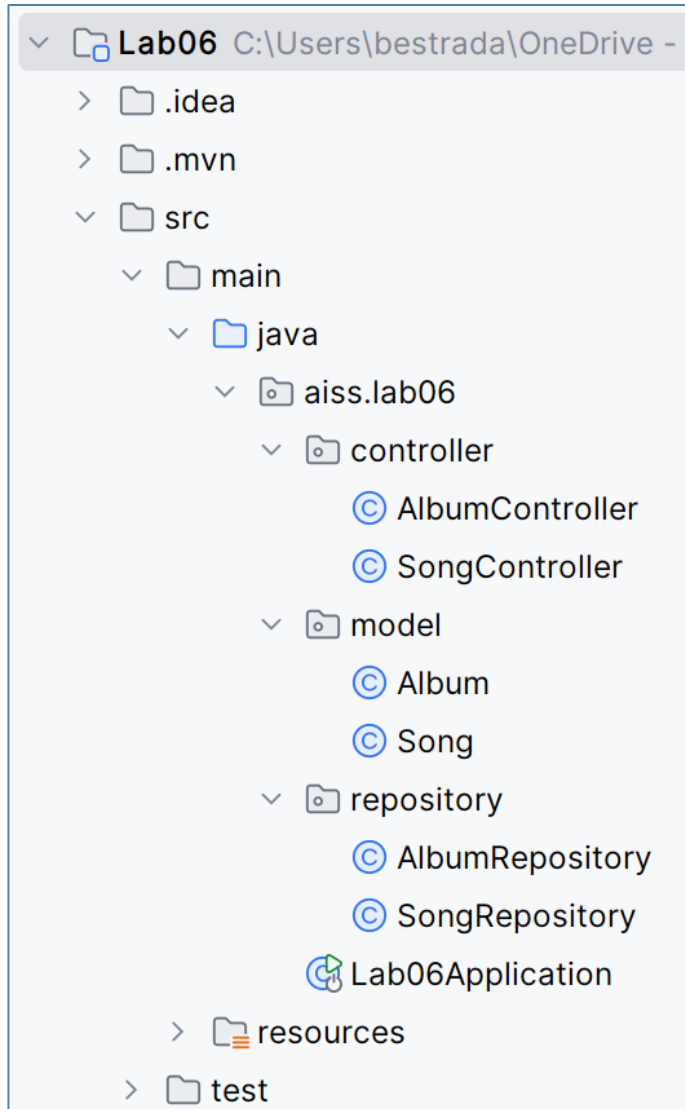
HTTP	URI	Descripción
GET	/api/albums	Lista todos los álbumes de la aplicación
GET	/api/albums/{albumId}	Devuelve el álbum con id=albumId
POST	/api/albums	Crea un nuevo album
PUT	/api/albums/{albumId}	Actualiza el álbum con id=albumId
DELETE	/api/albums/{albumId}	Elimina el álbum con id=albumId

Recurso Song

HTTP	URI	Descripción
GET	/api/albums/{albumId}/songs	Lista todas las canciones del álbum con id=albumId
GET	/api/songs/{songId}	Devuelve la canción con id=songId
POST	/api/albums/{albumId}/songs	Crea una nueva canción en el álbum con id=albumId
PUT	/api/songs/{songId}	Actualiza la canción con id=songId
DELETE	/api/songs/{songId}	Elimina la canción con id=songId

Estructura del proyecto de ejemplo

Estructura del proyecto de Spring Boot

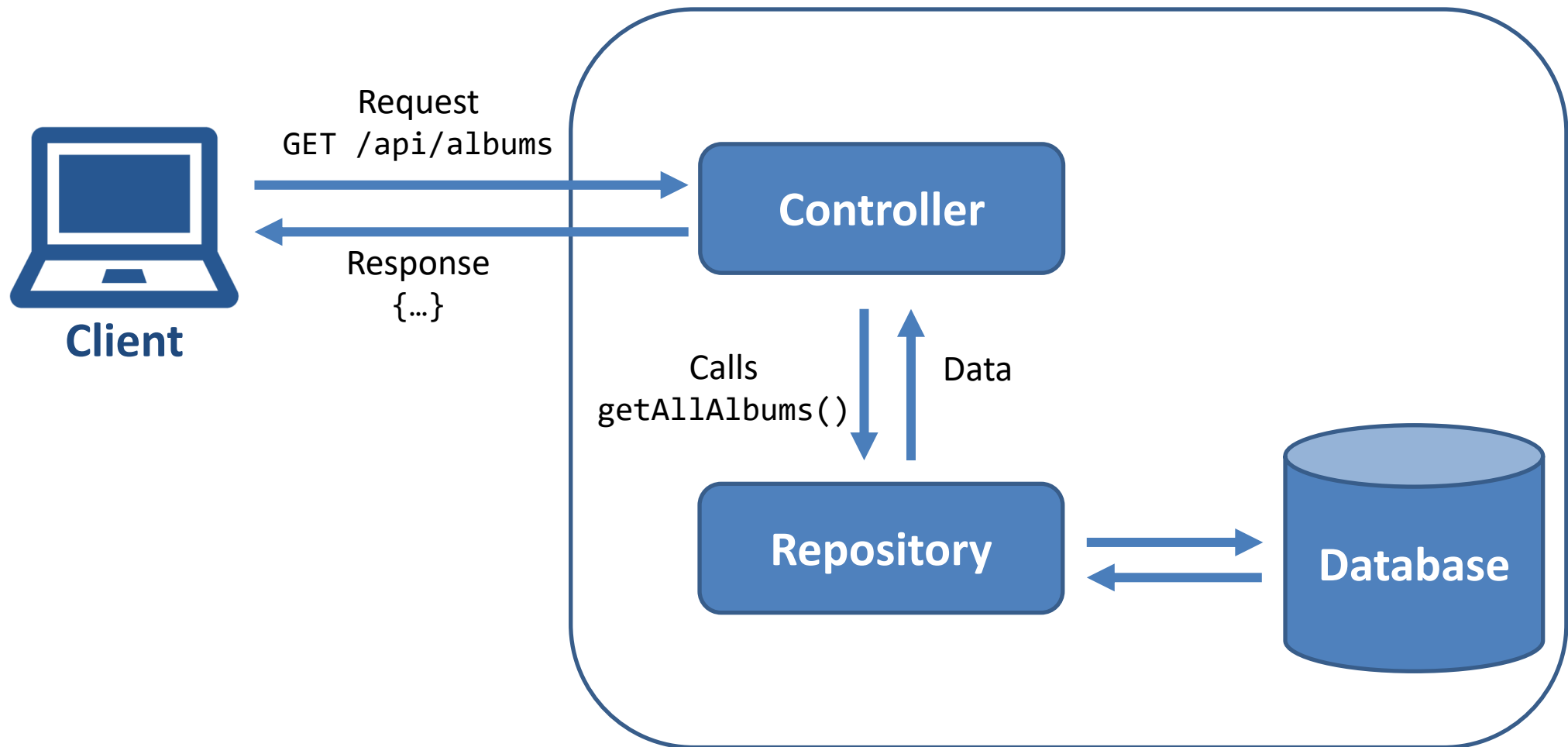


- **Paquete model:** Contiene las clases que representan los datos de la aplicación. Cada clase representa una tabla de la base de datos. Generalmente, tendremos una clase por recurso.
- **Paquete repository.** Contiene las interfaces que se encargan de interactuar con la base de datos. Suelen contener métodos para realizar las operaciones CRUD (crear, leer, actualizar y borrar).
- **Paquete controller.** Contiene las clases que se encargan de gestionar las solicitudes HTTP que llegan a la aplicación. Los controladores suelen interactuar con los servicios/repositorios para obtener y almacenar datos.

Estructura del proyecto de ejemplo

Estructura del proyecto de Spring Boot

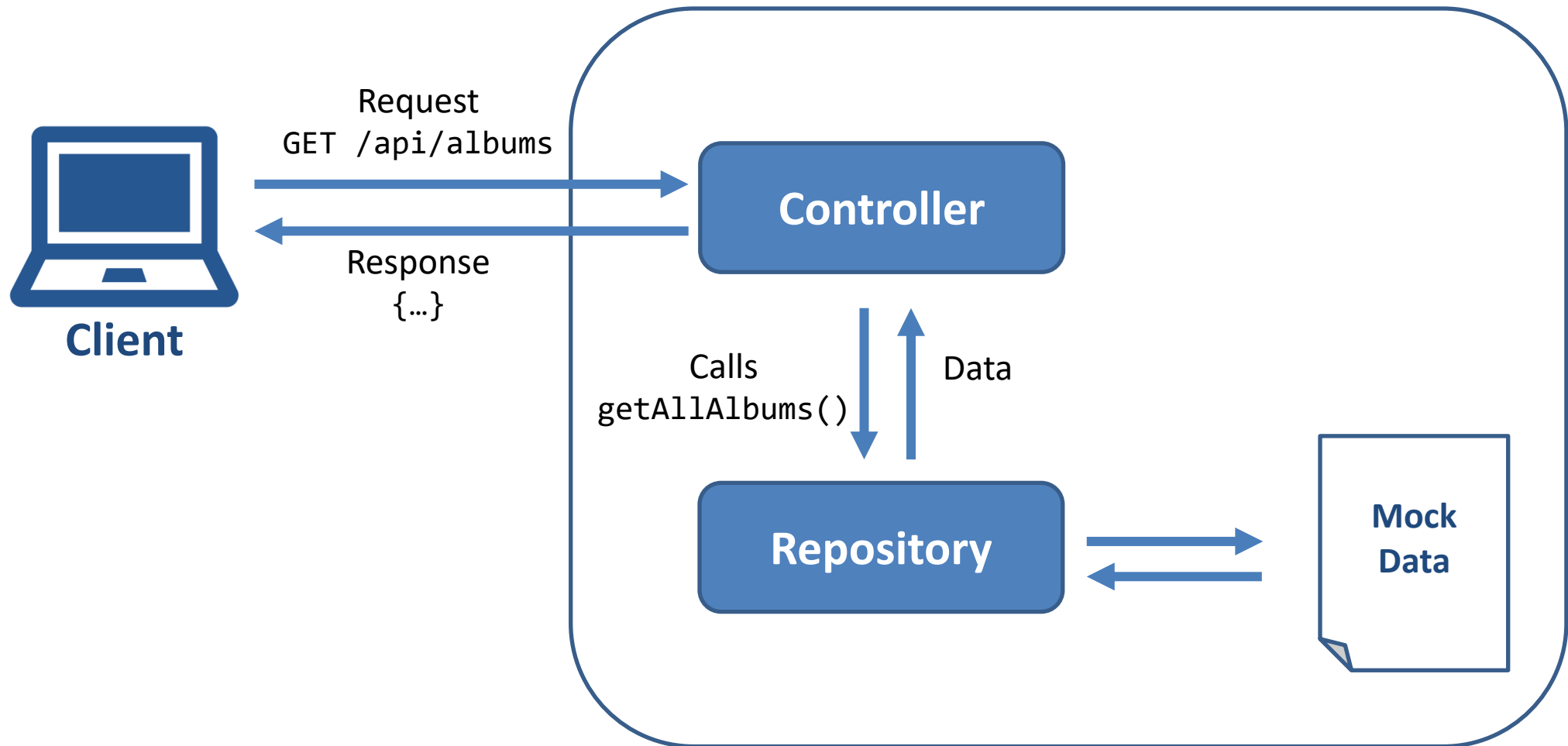
Server (Spring Boot API)



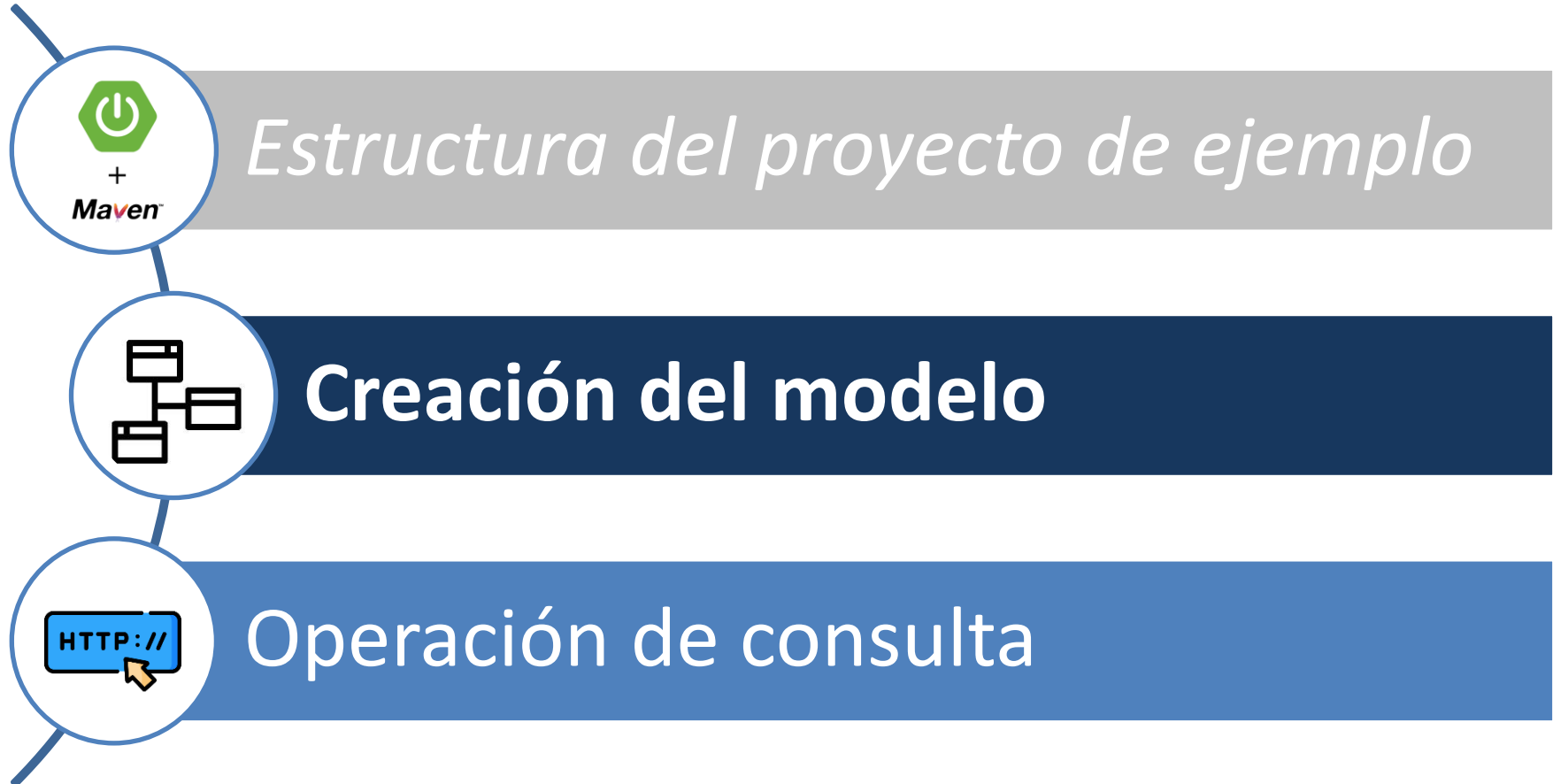
Estructura del proyecto de ejemplo

Estructura del proyecto de Spring Boot

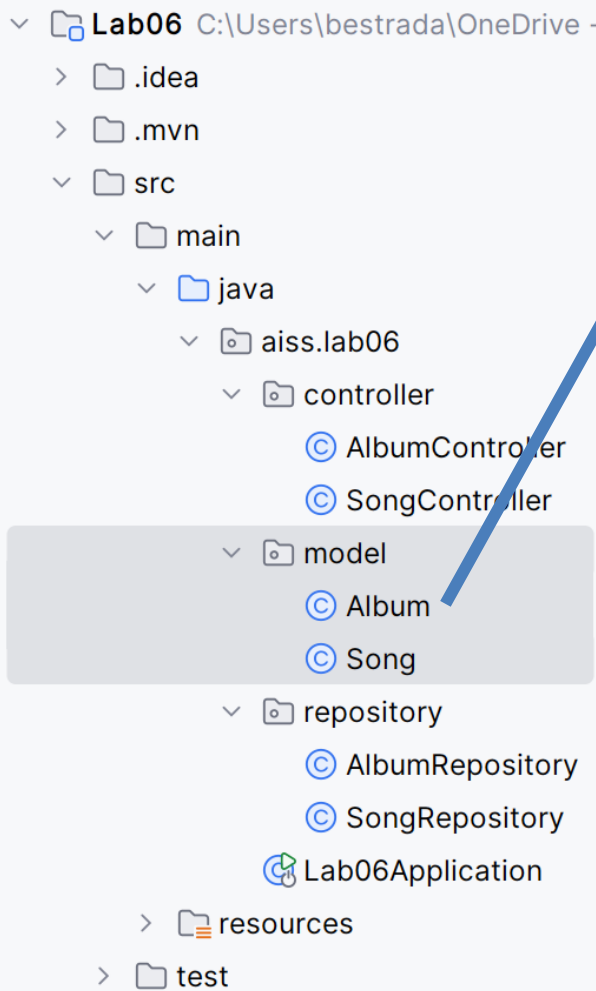
Server (Spring Boot API)



Índice – Vídeo 1

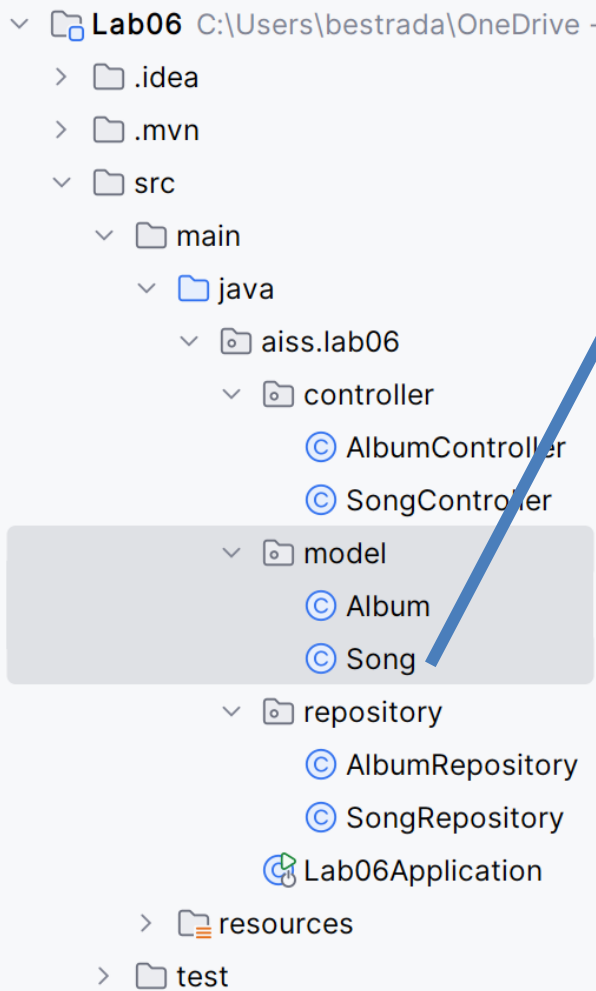


Creación del modelo



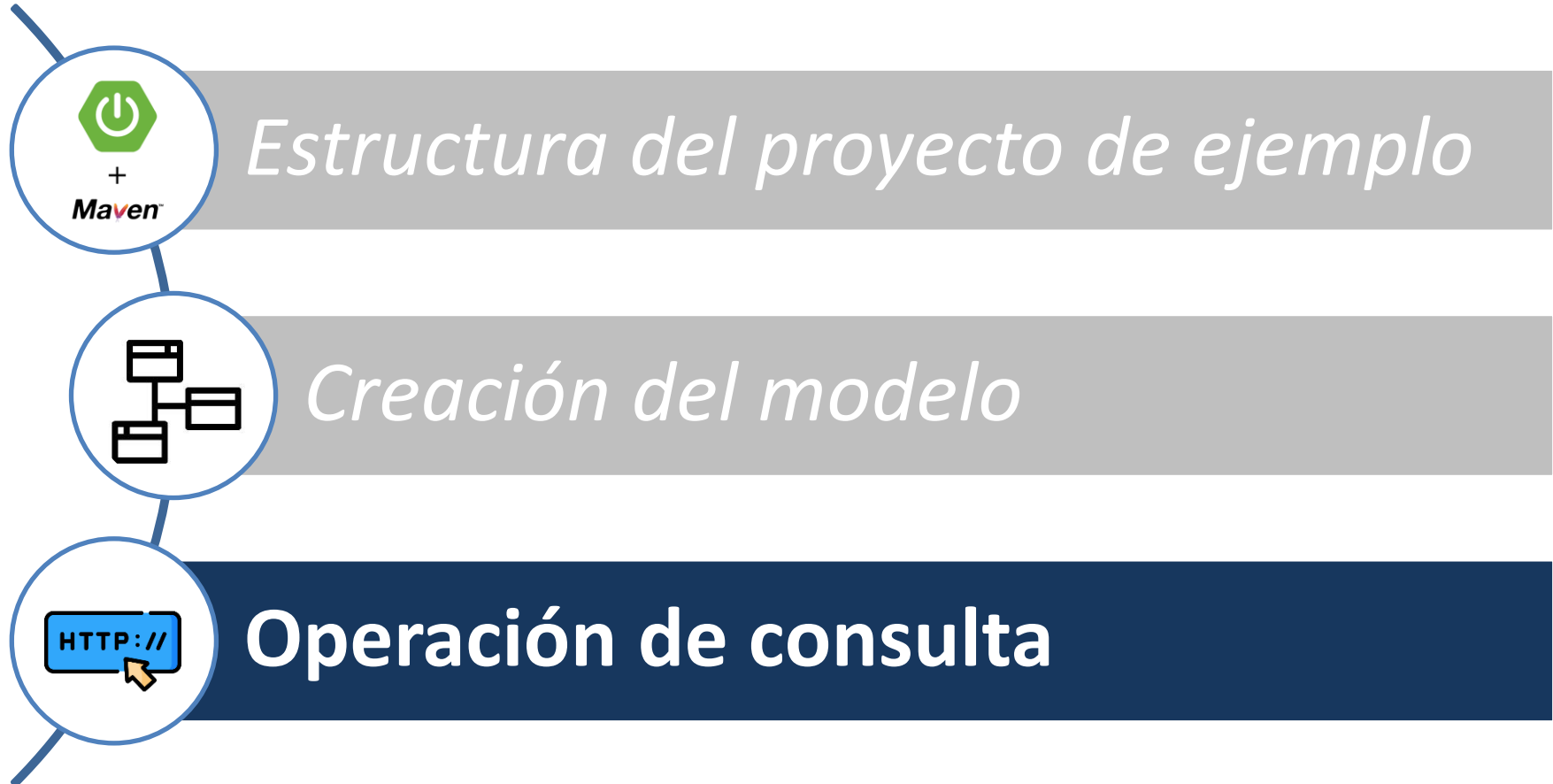
```
public class Album {  
  
    private String id;  
    private String name;  
    private String description;  
    private List<Song> songs;  
  
    public Album(String id, String name, String description) {  
        this.id = id;  
        this.name = name;  
        this.description = description;  
        this.songs = new ArrayList<Song>();  
    }  
  
    public String getId() { return id; }  
  
    public void setId(String id) { this.id = id; }  
  
    public String getName() { return name; }
```

Creación del modelo



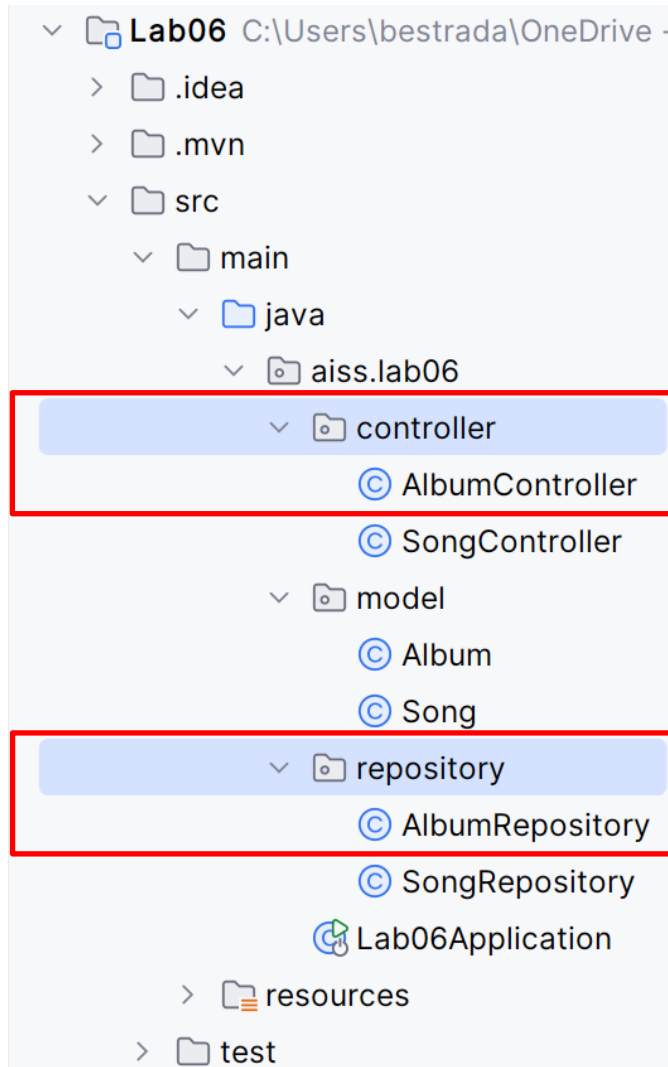
```
public class Song {  
  
    private String id;  
    private String title;  
    private LocalDateTime published;  
    private Integer trackNumber;  
    private Boolean playable;  
  
    public Song(String id, String title, LocalDateTime published,  
                Integer trackNumber, Boolean playable) {  
        this.id = id;  
        this.title = title;  
        this.published = published;  
        this.trackNumber = trackNumber;  
        this.playable = playable;  
    }  
  
    public String getId() { return id; }  
  
    public void setId(String id) { this.id = id; }
```

Índice – Vídeo 1



Operaciones de consulta

Creación del repositorio y controlador



@RestController

Rutas de acceso a los recursos
Llamadas al repositorio

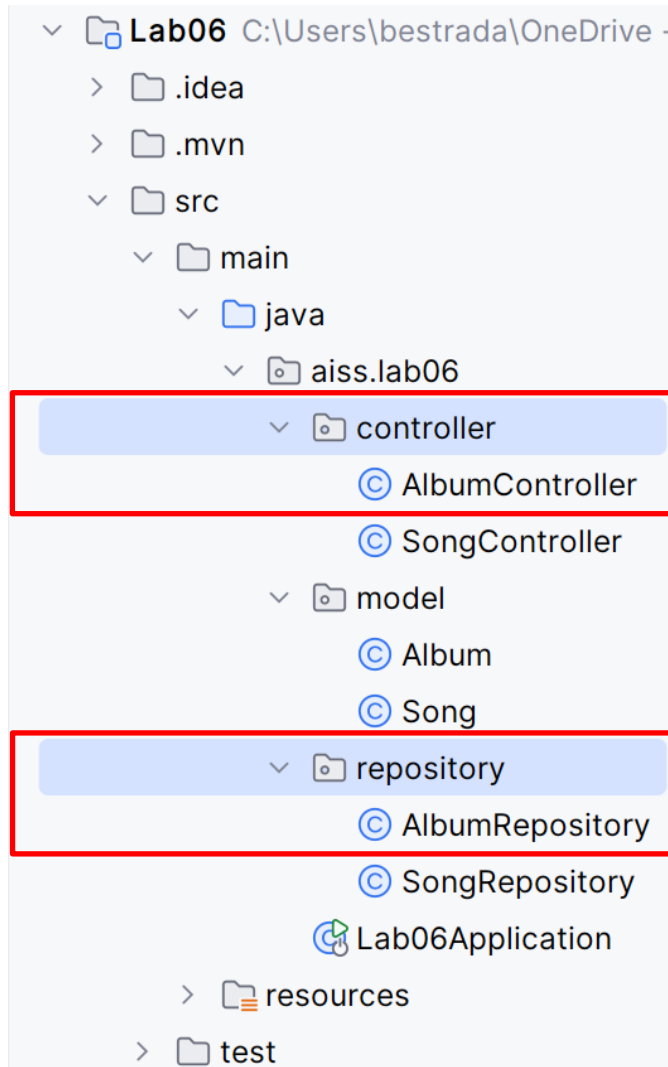


@Repository

Datos de prueba
mientras no usemos la base de datos

Operaciones de consulta

Creación del repositorio y controlador



@RestController

Rutas de acceso a los recursos
Llamadas al repositorio



@Repository

Datos de prueba
mientras no usemos la base de datos

Operaciones de consulta

Creación del repositorio (Datos de prueba)

Anotación para indicar que la clase es un repositorio

`@Repository`

```
public class AlbumRepository {  
  
    List<Album> albums = new ArrayList<>();  
  
    //Datos de ejemplo porque no estamos usando base de datos  
    public AlbumRepository() {  
        albums.add(  
            new Album(  
                UUID.randomUUID().toString(),  
                name: "A Night at the Opera",  
                description: "Album description"  
            )  
        );  
        albums.add(  
            new Album(  
                UUID.randomUUID().toString(),  
                name: "Nevermind",  
                description: "Album description 2"  
            )  
        );  
    }  
}
```

Definimos datos de prueba

Operaciones de consulta

Creación del repositorio (Operaciones)

Dentro de la misma clase `@Repository`
`public class AlbumRepository {` agregamos los métodos de consulta.

- Para devolver todos los álbumes de la aplicación.

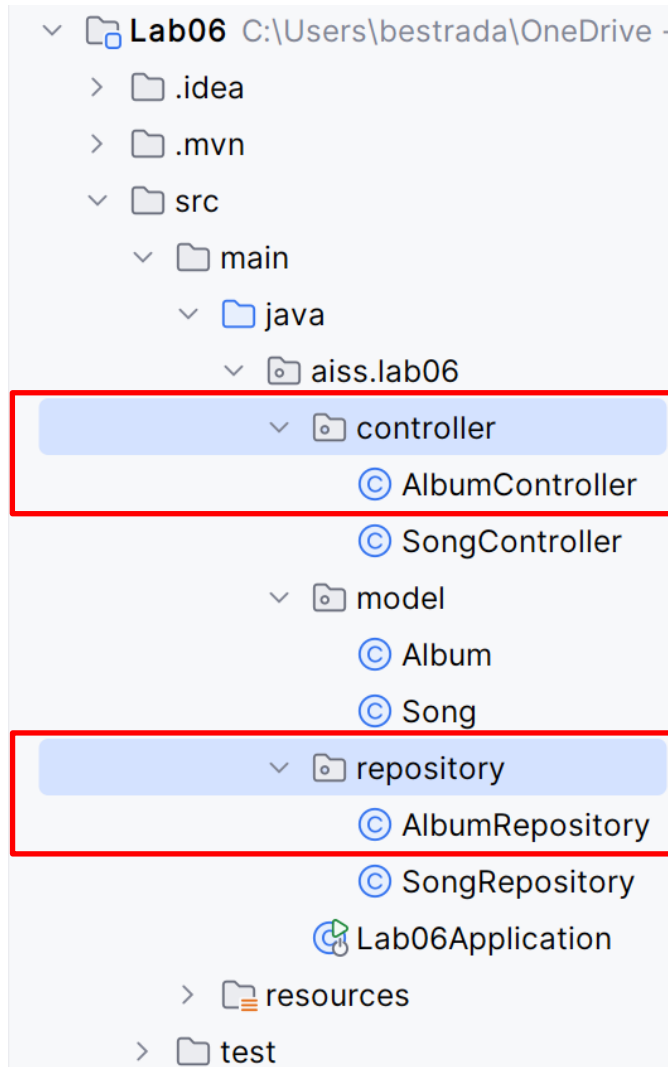
```
public List<Album> findAll() { return albums; }
```

- Para devolver el álbum con id=albumId

```
public Album findOneById(String id) {  
    return albums.stream() Stream<Album>  
        .filter( Album album ->album.getId().equals(id))  
        .findFirst() Optional<Album>  
        .orElse( other: null);  
}
```

Operaciones de consulta

Creación del repositorio y controlador



@RestController

Rutas de acceso a los recursos
Llamadas al repositorio



@Repository

Datos de prueba
mientras no usemos la base de datos

Operaciones de consulta

Creación de controladores

Anotación para indicar que la clase es un controlador

Anotación para definir la ruta base de las operaciones del controlador

```
@RestController
@RequestMapping("/api/albums")
public class AlbumController {

    private final AlbumRepository albumRepository;

    @Autowired
    public AlbumController(AlbumRepository albumRepository) {
        this.albumRepository = albumRepository;
    }
}
```

Inicializamos el repositorio con los datos de prueba

Operaciones de consulta

Creación de controladores (Operaciones)

- Para devolver todos los álbumes de la aplicación

Especificamos el verbo HTTP a utilizar

```
//GET http://localhost:8080/api/albums  
@GetMapping  
public List<Album> findAll(){  
    return albumRepository.findAll();  
}
```

Llamamos a un método del repositorio (definido en diapositiva anterior)

- Para devolver el álbum con id=albumId

Podemos especificar rutas más concretas, que se añaden al valor especificado en RequestMapping

```
//GET http://localhost:8080/api/albums/{id}  
@GetMapping("/{id}")  
public Album findById(@PathVariable String id) {  
    return albumRepository.findOneById(id);  
}
```

Indicamos que el parámetro "id" se envía por el path



Departamento de Lenguajes y
Sistemas Informáticos

Vídeo 2

Implementación de Servicios RESTful (I)

Integración



Arquitectura e Integración del Software

Contenido de la práctica

■ **Vídeo 1:**

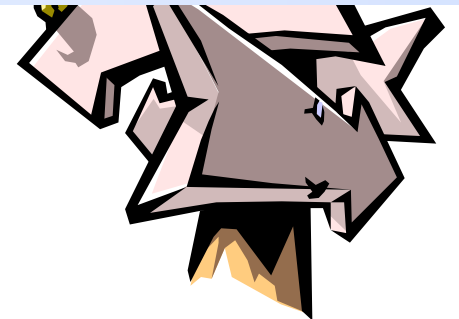
- Estructura del proyecto de ejemplo
- Creación del modelo
- Operaciones de consulta:

■ **Vídeo 2:**

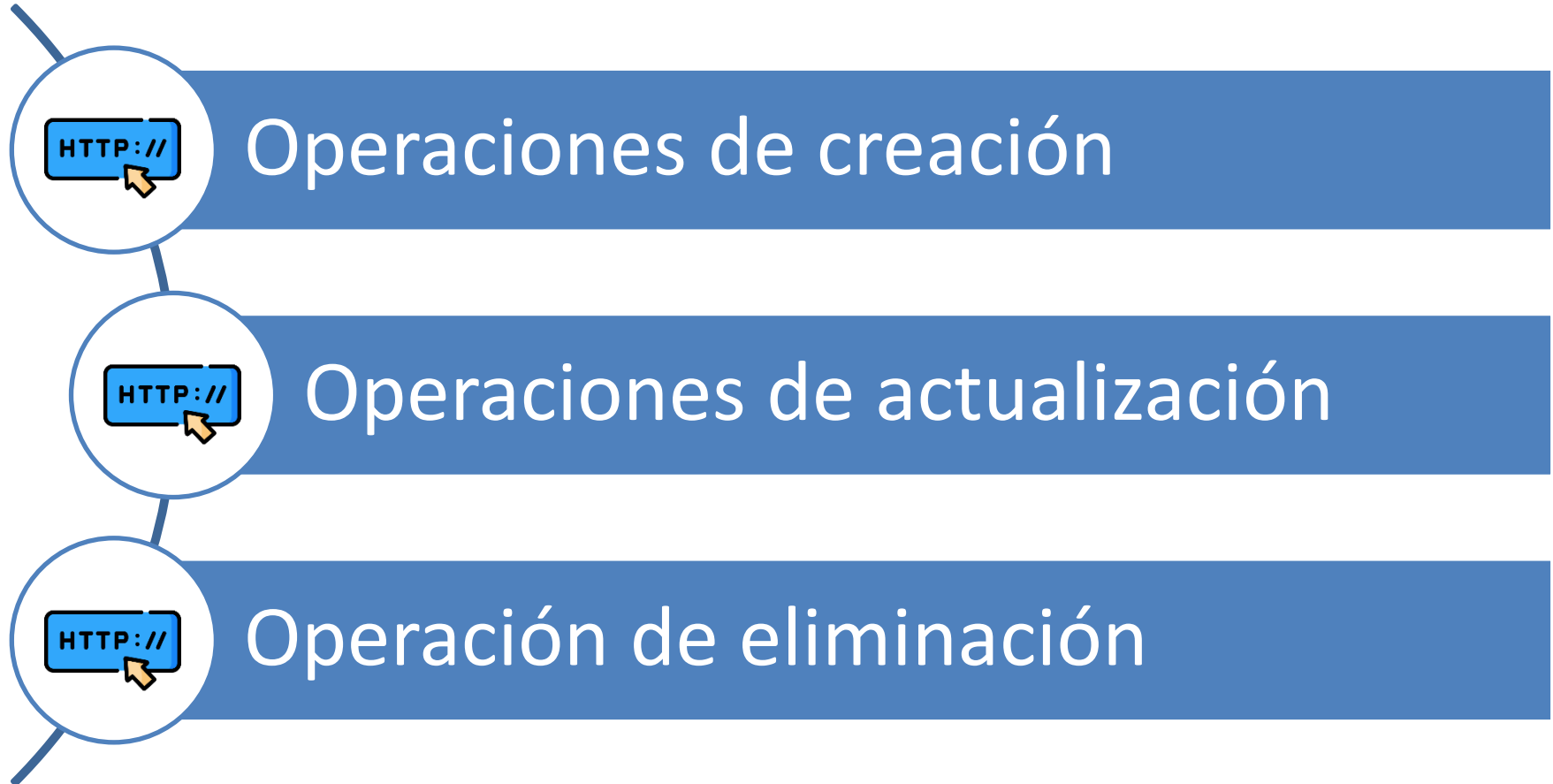
- Operaciones de creación
- Operaciones de actualización
- Operaciones de eliminación

■ **Vídeo 3:**

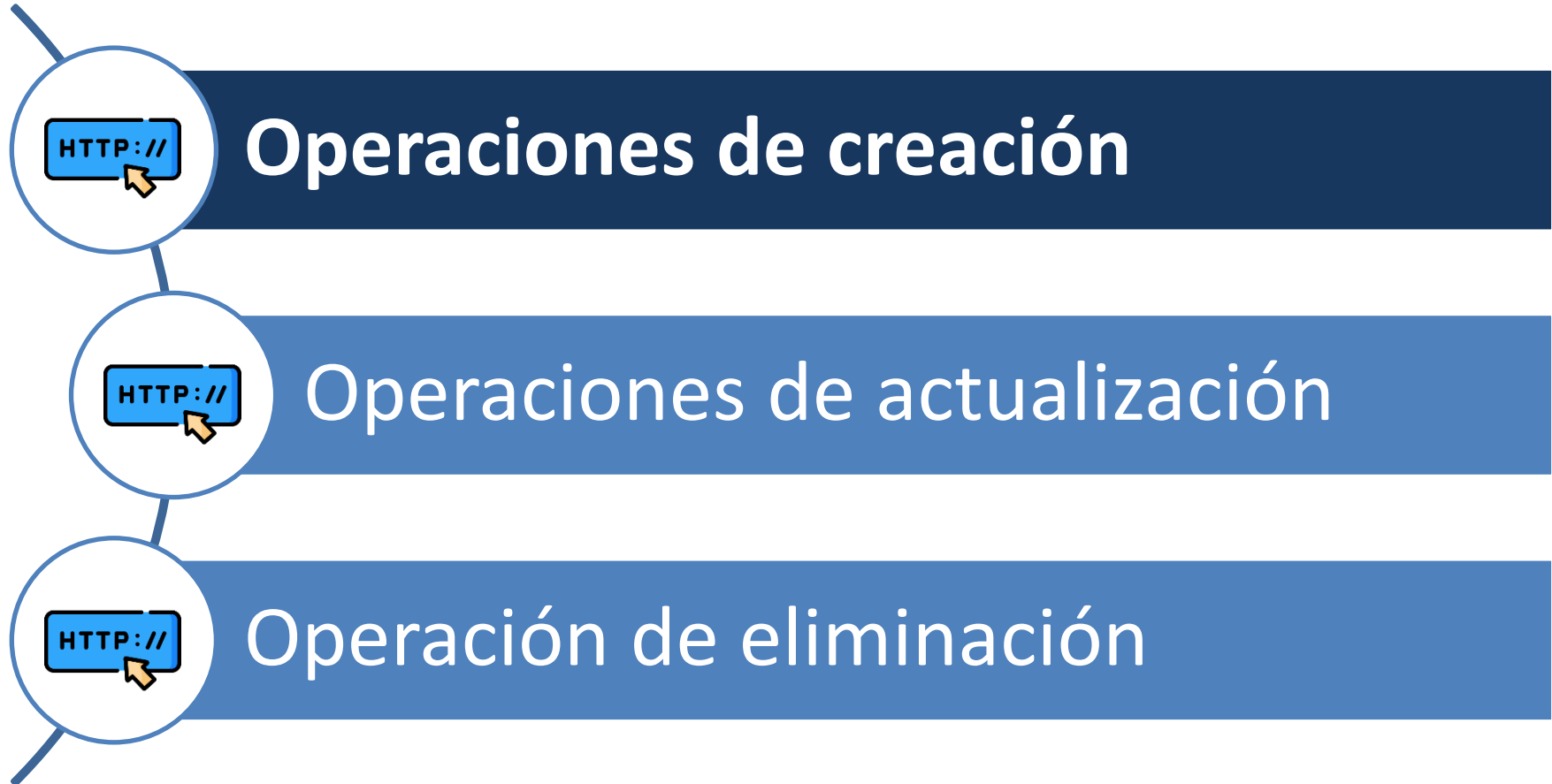
- Pruebas



Índice – Vídeo 2



Índice – Vídeo 2



Operaciones de creación

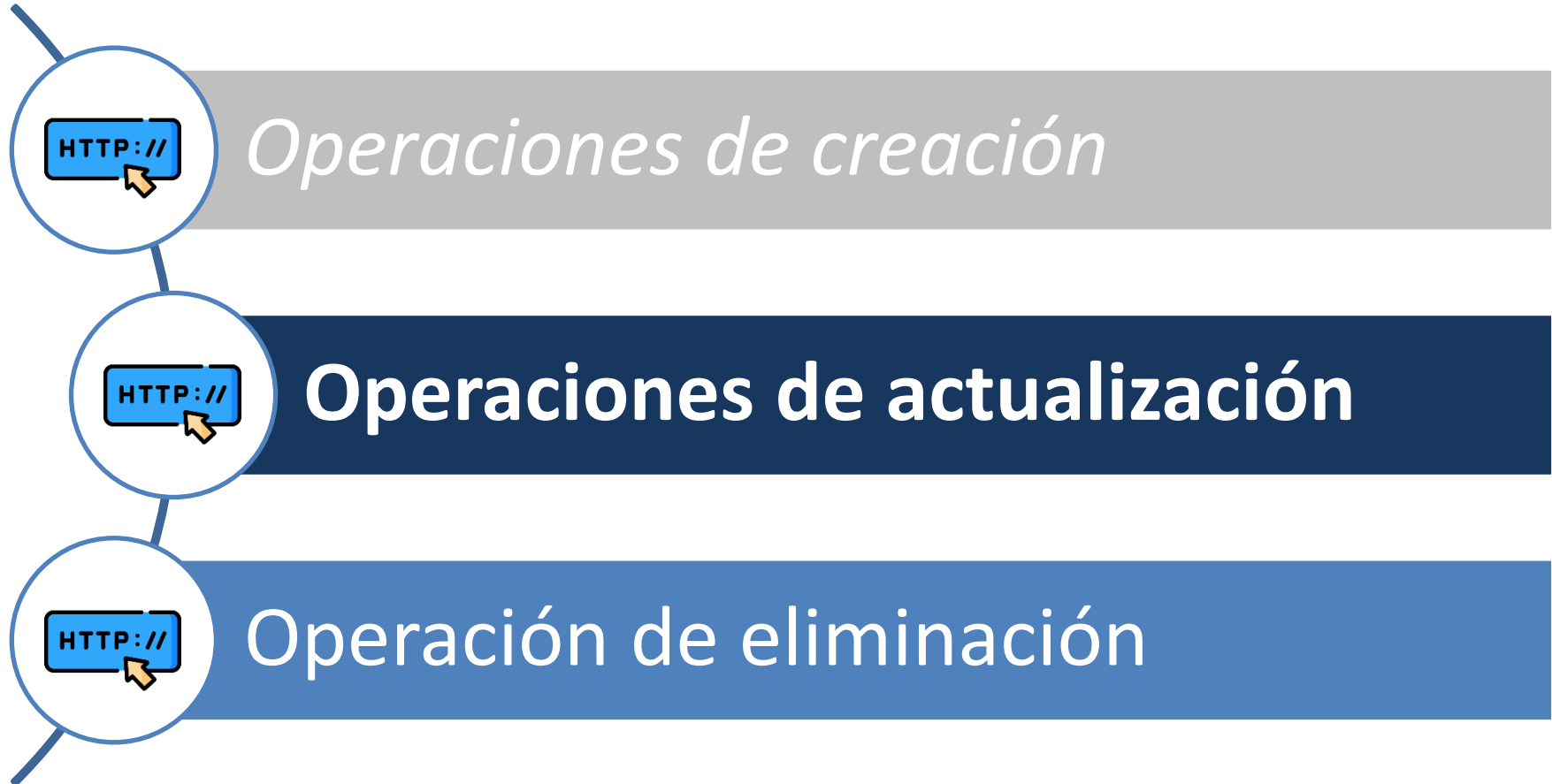
- **Repositorio:** Implementamos un método para crear un nuevo álbum.

```
public Album create(Album album) {  
    Album newAlbum = new Album(  
        UUID.randomUUID().toString(),  
        album.getName(),  
        album.getDescription()  
    );  
    albums.add(newAlbum);  
    return newAlbum;  
}
```

- **Controlador:** Implementamos un método indicando que se trata de una operación de creación (@PostMapping) que recibe un álbum válido (@Valid) en el cuerpo de la petición (@RequestBody) y devuelve un código 201 (@ResponseStatus(HttpStatus.CREATED)).

```
//POST http://localhost:8080/api/albums  
@ResponseStatus(HttpStatus.CREATED) //201  
@PostMapping  
public Album create(@Valid @RequestBody Album album) { return albumRepository.create(album); }
```

Índice – Vídeo 2



Operaciones de actualización

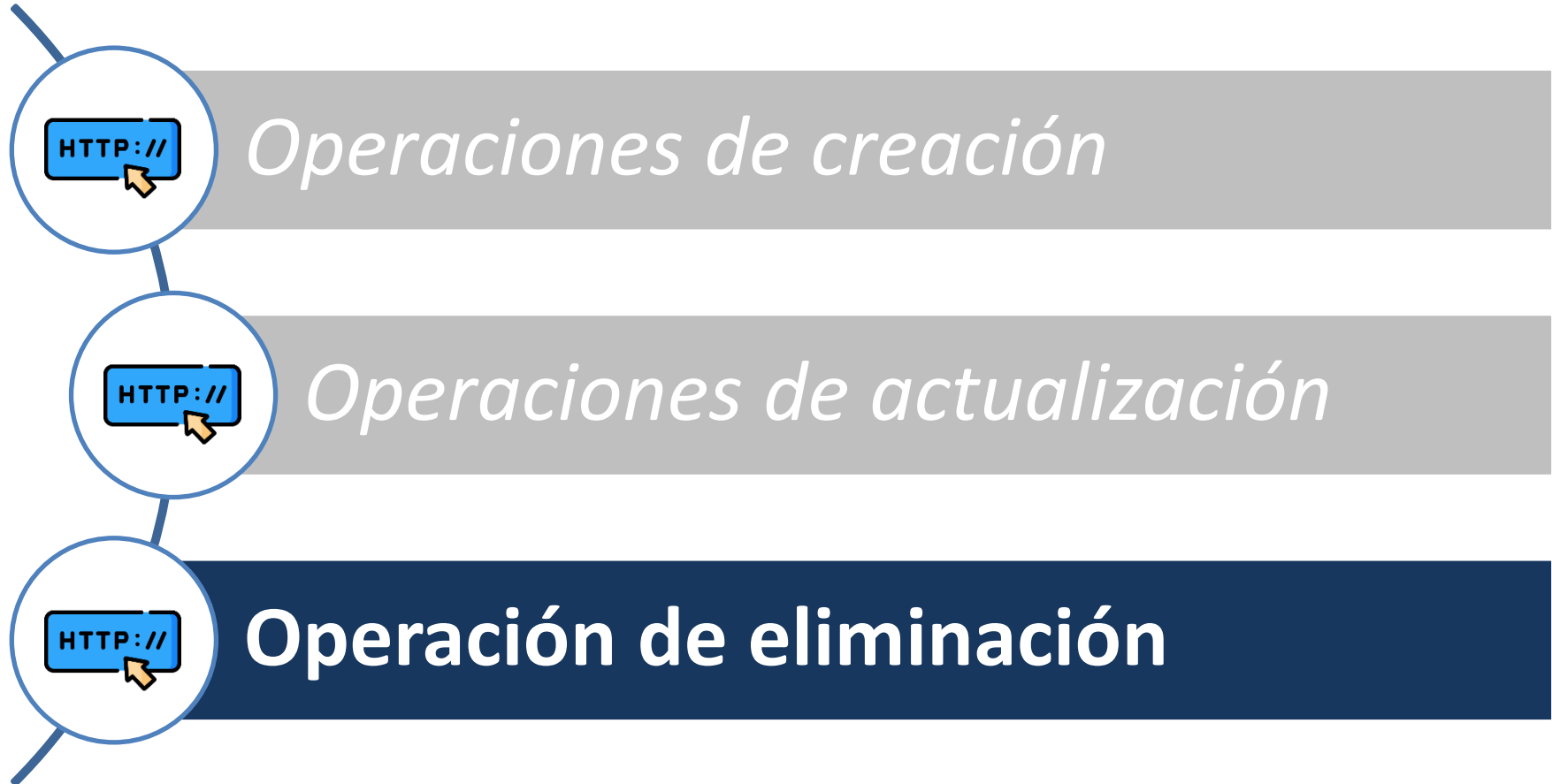
- **Repositorio:** Implementamos un método para actualizar un álbum existente dado su id.

```
public void update(Album updatedAlbum, String id) {  
    Album existingAlbum = findOneById(id);  
    int i = albums.indexOf(existingAlbum);  
    updatedAlbum.setId(existingAlbum.getId());  
    albums.set(i, updatedAlbum);  
}
```

- **Controlador:** Implementamos un método indicando que se trata de una operación de actualización (@PutMapping) que recibe el id del álbum a actualizar como parámetro del path (@PathVariable) y los datos del álbum a actualizar en el cuerpo de la petición (@RequestBody). Devuelve un código 204 (@ResponseStatus(HttpStatus.NO_CONTENT)).

```
//PUT http://localhost:8080/api/albums/{id}  
@ResponseStatus(HttpStatus.NO_CONTENT)  
@PutMapping("/{id}")  
public void update(@Valid @RequestBody Album updatedAlbum, @PathVariable String id) {  
    albumRepository.update(updatedAlbum, id);  
}
```

Índice – Vídeo 2



Operaciones de eliminación

- **Repositorio.** Implementamos un método para eliminar un álbum dado su id.

```
public void delete(String id) {  
    albums.removeIf(album -> album.getId().equals(id));  
}
```

- **Controlador.** Implementamos un método indicando que se trata de una operación de eliminación (`@DeleteMapping`) que recibe el id del álbum a eliminar como parámetro del path (`@PathVariable`) y devuelve un código 204 (`@ResponseStatus(HttpStatus.NO_CONTENT)`).

```
//DELETE http://localhost:8080/api/albums/{id}  
@ResponseStatus(HttpStatus.NO_CONTENT)  
@DeleteMapping("/{id}")  
public void delete(@PathVariable String id) {  
    albumRepository.delete(id);  
}
```



Departamento de Lenguajes y
Sistemas Informáticos

Vídeo 3

Implementación de Servicios RESTful (I)

Integración



Arquitectura e Integración del Software

Contenido de la práctica

- **Vídeo 1:**
 - Estructura del proyecto de ejemplo
 - Creación del modelo
 - Operaciones de consulta:
- **Vídeo 2:**
 - Operaciones de creación
 - Operaciones de actualización
 - Operaciones de eliminación
- **Vídeo 3:**
 - Pruebas



Pruebas

Tests en Postman

The screenshot displays the Postman interface for a GET request to `http://localhost:8080/api/albums`. The **Scripts** tab is active, showing two test scripts:

```
1 pm.test("Status code is 200", function () {  
2   | pm.response.to.have.status(200);  
3   | });  
4  
5 pm.test("All the elements of the response must contain id property", function() {  
6   | var jsonData = pm.response.json();  
7   | for(var i=0; i<jsonData.length; i++){  
8   |   | pm.expect(jsonData[i]).to.have.property("id");  
9   | }  
10  | })
```

The **Post-response** tab is also visible in the left sidebar. The **Test Results (2/2)** section at the bottom shows two passed tests:

- PASSED** Status code is 200
- PASSED** All the elements of the response must contain id property

The status bar at the bottom indicates a **200 OK** response with a 3 ms duration and 398 B of data.

Pruebas

Estructura de un test en Postman

Nombre del test

Comprobaciones {

```
1 pm.test("Status code is 200", function () {  
2 |   pm.response.to.have.status(200);  
3 | });
```

Pruebas

Ejemplos (1)

- Comprobar el código de respuesta.

```
1 pm.test("Status code is 200", function () {  
2   |   pm.response.to.have.status(200);  
3   });
```

- Comprobar que el cuerpo de la respuesta contiene un string específico.

```
1 pm.test("The list of songs is empty", function () {  
2   |   pm.expect(pm.response.text()).to.include("\"songs\":[]");  
3   });
```

Pruebas

Ejemplos (2)

- Comprobar que el cuerpo de la respuesta contiene una propiedad.

```
pm.test("Response contains id property", function () {  
    var jsonData = pm.response.json();  
    pm.expect(jsonData).to.have.property("id");  
})
```

- Comprobar que todos los elementos de un array tienen una propiedad.

```
pm.test("All array elements should have the id property", function() {  
    var jsonData = pm.response.json();  
    for(var i=0; i<jsonData.length; i++) {  
        pm.expect(jsonData[i]).to.have.property("id");  
    }  
})
```

Pruebas

Recursos adicionales

- Vídeo tutorial oficial: https://www.youtube.com/watch?v=oXW-C2bM0wE&ab_channel=Postman
- Documentación oficial: <https://learning.postman.com/docs/writing-scripts/test-scripts/>
- Ejecutar colecciones de Postman: <https://learning.postman.com/docs/collections/running-collections/intro-to-collection-runs/>
- Escribir pre-request scripts: <https://learning.postman.com/docs/writing-scripts/pre-request-scripts/>
- Más ejemplos de scripts de pruebas: <https://learning.postman.com/docs/writing-scripts/script-references/test-examples/>
- Colección de Postman con ejemplos avanzados: <https://www.postman.com/postman/workspace/test-examples-in-postman/documentation/1559645-820d771d-70ab-452f-9edd-0904dbc315b8>

Disclaimer and Terms of Use

All material displayed on this presentation is for teaching and personal use only.

Many of the images that have been used in the presentation are Royalty Free images taken from <http://www.everystockphoto.com/>. Other images have been sourced directly from the Public domain, from where in most cases it is unclear whether copyright has been explicitly claimed. Our intention is not to infringe any artist's copyright, whether written or visual. We do not claim ownership of any image that has been freely obtained from the public domain. In the event that we have freely obtained an image or quotation that has been placed in the public domain and in doing so have inadvertently used a copyrighted image without the copyright holder's express permission we ask that the copyright holder writes to us directly, upon which we will contact the copyright holder to request full written permission to use the quote or images.



Departamento de Lenguajes y
Sistemas Informáticos

Vídeo 3

Implementación de Servicios RESTful (I)

Integración



Arquitectura e Integración del Software

Disclaimer and Terms of Use

All material displayed on this presentation is for teaching and personal use only.

Many of the images that have been used in the presentation are Royalty Free images taken from <http://www.everystockphoto.com/>. Other images have been sourced directly from the Public domain, from where in most cases it is unclear whether copyright has been explicitly claimed. Our intention is not to infringe any artist's copyright, whether written or visual. We do not claim ownership of any image that has been freely obtained from the public domain. In the event that we have freely obtained an image or quotation that has been placed in the public domain and in doing so have inadvertently used a copyrighted image without the copyright holder's express permission we ask that the copyright holder writes to us directly, upon which we will contact the copyright holder to request full written permission to use the quote or images.